

jordanvalnet/exalthon26

Onboard : une chaîne à validations dures, mais 2 fichiers de test, aucune CI et tout écrit en une semaine

Hackathon eXaltemps 2026 : agent + MCP GitHub

2

FICHIERS DE TEST

bun:test

aucune

INTÉGRATION CONTINUE

1

ISSUES OUVERTES

<https://github.com/jordanvalnet/exalthon26> Sans licence Onboard · profil qa

☑ Que fait ce projet et quels sont ses parcours critiques ?

VERDICT

Cinq parcours à qualifier, dont deux dépendent d'un service extérieur : le serveur MCP GitHub et Claude Code.

Onboard lit n'importe quel dépôt GitHub par le serveur MCP GitHub, construit un cache documentaire vérifié et en tire une présentation par public : dev, qa, cto, ceo, investisseur, enfant.

Une commande, `bun onboard owner/repo profil`, enchaîne trois étapes, collecter, rédiger, rendre, puis un guide répond aux questions à partir du cache ; chaque étape ne démarre que si la validation en code de la précédente passe.

Créé le 9 septembre 2026 pour le hackathon « Agent + MCP GitHub » de l'équipe eXaltemps, dernier push le 10 septembre ; aucune release, 0 étoile, 0 fork.

Parcours critiques à qualifier, dans l'ordre où ils cassent le plus probablement :

- 1 Accès GitHub : `GITHUB_PAT` dans `.env`, serveur MCP déclaré dans `.mcp.json` ; `bun run dev` affiche « GitHub OK : <login> » ou sort en code 1.
- 2 Collecte : `bun run meta` et `bun run issues`, puis `bun run merge` qui assemble `parts/*.json` dans `facts.json`.
- 3 Validations : `bun run validate facts|narrative|deck`, la barrière entre deux étapes.
- 4 Rendu : `bun run render` puis `bun run pdf`, qui exige Chrome ou Edge en headless.
- 5 Orchestration : `bun onboard` lance Claude Code en non interactif et exige sa présence dans le PATH.

☑ Comment est-il testé aujourd'hui ?

VERDICT

2 fichiers de test pour 15 scripts : aujourd'hui, la barrière principale est le typecheck, pas les tests.

- ☐ `.claude/` skills Claude Code du pipeline
- ☐ `chat/` client du chat d'équipe
- ☐ `docs/` documentation du projet
- ☐ `onboard/` pipeline : docs, agents, profils, caches
- ☐ `onboarding/` sous-projet bun, collecteurs et rapport
- ☐ `pitch/` supports de pitch
- ☐ `src/` CLI, collecte, validation, rendu
- ☐ `test/` tests bun
- ☐ `.env.example` modèle du token GitHub
- ☐ `.gitignore` fichiers ignorés par git
- ☐ `.mcp.json` déclaration du serveur MCP GitHub
- ☐ `AGENTS.md` consignes projet pour les IA
- ☐ `CIBLES.md` repos cibles du hackathon
- ☐ `CLAUDE.md` renvoi vers AGENTS.md
- ☐ `HACKATHON.md` consignes du jury
- ☐ `LIVRABLE.md` rapport de mission de l'équipe
- ☐ ... et 5 autres entrées à la racine du dépôt

Un dossier `test/` à la racine, lanceur `bun test`, 2 fichiers : `test/github.test.ts` (801 octets) et `test/issues.test.ts` (1654 octets).

La commande de référence est `bun run check = tsc --noEmit` puis `bun test` : le typecheck pèse autant que les tests.

À en juger par leurs noms, les deux fichiers visent l'accès GitHub et le collecteur d'issues ; leur contenu n'a pas été lu.

Ce que ces noms ne couvrent pas, sur les 9 entrées de `src/` et les 15 scripts du manifeste :

- ☐ `src/validate.ts` (8324 octets) : les trois validations dures du pipeline
- ☐ `src/merge.ts` (3849 octets) : la fusion des collectes dans `facts.json`
- ☐ `src/render/` : deck HTML, PDF, capture d'écran
- ☐ `src/cli.ts` (2530 octets) : l'orchestration via Claude Code
- ☐ `onboarding/` : second package bun, manifeste et code non lus
- ☐ `chat/chat.mjs` : le client du chat d'équipe, node ou bun

☑ Que dit la CI et quand tourne-t-elle ?

VERDICT

Rien ne tourne sur GitHub : un push qui casse `bun run check` n'est détecté par personne tant qu'un poste ne le relance pas.

Rien : `.github/workflows/` renvoie 404 et la racine, 21 entrées, ne contient aucun dossier `.github` .

La seule vérification est locale, `bun run check` , que le README demande de lancer « avant de pousser » ; elle repose sur la discipline de chaque personne.

La collecte note ce risque mid : aucun workflow GitHub Actions, aucune vérification hébergée.

Conséquence pour la qualification : aucune trace d'exécution des tests n'existe sur GitHub ; il faut les rejouer soi-même sur le commit à qualifier.

Rien ne contrôle non plus les dépendances : `@types/bun` est en `latest` , non épinglée, malgré un `bun.lock` versionné.

📌 Où sont les bugs connus ?

VERDICT

Aucun bug tracé : le tracker est vide parce que personne n'a signalé, pas parce que le produit est qualifié.

Nulle part dans le tracker : 1 issue ouverte, 0 fermée sur 30 jours, aucun label, donc aucun ticket « bug » et aucune good first issue.

L'unique issue ouverte, #1 « Chat équipe — canal IA ↔ IA », 22 commentaires, est le canal où les 6 assistants IA de l'équipe se parlent, pas un défaut¹.

Un tracker vide sur un projet d'une semaine ne dit rien de la qualité : personne n'a encore signalé. La liste des risques collectés est le seul inventaire disponible.

RISQUE	NIVEAU	CONSTAT
Licence	HIGH	pas de fichier LICENSE, réutilisation juridiquement incertaine
Bus factor	HIGH	un seul contributeur porte la moitié des commits sur 12 semaines
CI	MID	aucun workflow, vérification locale seulement
Sécurité	MID	pas de SECURITY.md, aucun canal pour signaler une faille
Dépendances	LOW	3 déclarées, @types/bun non épinglée

Aucun commentaire T0D0 de dette dans src/ ni test/ : la recherche renvoie 12 résultats, tous dans de la documentation, des prompts ou des caches générés.

☑ Comment reproduire et signaler un bug ici ?

VERDICT

Pas de CONTRIBUTING et pas de label bug : un défaut se signale par une issue neuve, jamais dans #1.

Donnée non relevée : CONTRIBUTING.md ; le fichier est absent des 21 entrées de la racine, à vérifier dans <https://github.com/jordanvalnet/exalthon26> ; aucune procédure de signalement n'est écrite.

Reproduire : l'environnement tient en quelques lignes du README, bun, `.env` avec `GITHUB_PAT`, `bun install && bun run check`, puis la commande d'onboarding.

```
cp .env.example .env          # GITHUB_PAT=<token GitHub, scope repo>
bun install && bun run check
bun onboard owner/repo qa
bun run validate facts|narrative|deck onboard/cache/<owner>__<repo> qa
```

Isoler : chaque étape se rejoue seule, `bun run merge`, `bun run validate`, `bun run render`, `bun run pdf`, et chaque sortie est un fichier texte versionné dans `onboard/cache/` : un diff montre ce qui a changé.

Comparer avec GitHub sans passer par le pipeline : `bun run gh <outil> '<args JSON>'` fait l'appel brut au serveur MCP, `bun run gh tools` liste les outils.

Signaler : 0 PR ouverte et 1 seule PR fusionnée, #2 le 9 septembre, pour 57 commits cette semaine ; les changements arrivent sur `main` sans passer par une PR².

L'issue #1 est réservée au chat des assistants et ne doit jamais être fermée ; un défaut se signale dans une issue neuve, avec l'étape (collecte, rédaction, rendu), le dossier de cache, le profil et la dernière ligne de la validation.

☑ Quelles zones sont peu couvertes ?

VERDICT

Le rendu, les validations et la fusion, soit le cœur du produit, n'ont aucun test connu ; tout a été écrit en une semaine, par une personne principale.

Tout le code a été écrit en une semaine : 57 commits la semaine 37, 1 la semaine 35, 0 les dix autres ; dernier commit le 10 septembre 2026.

Une personne signe 41 commits, plus que les quatre autres réunies (8, 4, 3, 2) ; bus factor 1, risque high.

Les 2 fichiers de test, à en juger par leurs noms, ne touchent que l'accès GitHub et les issues.

Zones sans test connu, par ordre de risque pour le livrable :

- ❑ `src/render/` : deck HTML, PDF via Chrome headless, capture ; c'est le livrable visible
- ❑ `src/validate.ts` : les trois barrières du pipeline ; une barrière qui laisse passer rend le reste inutile
- ❑ `src/merge.ts` : la fusion des `parts/*.json` dans `facts.json`
- ❑ `src/cli.ts` : l'orchestration via Claude Code, non testable sans l'agent
- ❑ `onboarding/` : second package bun, manifeste et code non lus par la collecte
- ❑ `chat/chat.mjs` : doit tourner sous node et sous bun

La documentation pour agents IA est en revanche bien présente, 4 repères sur 7 : AGENTS.md, CLAUDE.md, .mcp.json, .claude/skills/.



Plan de test en 5 points

VERDICT

Sans CI, chaque point de ce plan se rejoue à la main sur le commit à qualifier : le point 5 est celui qui rend les quatre autres durables.

- 1 Environnement : sur un poste neuf, `bun install` & `bun run check` passe et `bun run dev` affiche « GitHub OK : <login> » avec un `GITHUB_PAT` de scope repo.
- 2 Collecte : `bun run meta` et `bun run issues` sur un dépôt de `CIBLES.md`, comparer `parts/*.json` avec un appel brut `bun run gh`, vérifier que ce que le serveur MCP n'expose pas est déclaré manquant, jamais deviné.
- 3 Validations : corrompre volontairement `facts.json`, une citation du narratif et le deck, puis vérifier que `bun run validate facts|narrative|deck` bloque à chaque fois.
- 4 Rendu : `bun run render` puis `bun run pdf` pour les six profils sur un même cache, relecture à l'œil de chaque deck, aucun chiffre sans note de bas de page.
- 5 Non-régression : épingler `@types/bun` et ajouter un workflow qui lance `bun run check` à chaque push, aujourd'hui inexistant.

RÉFÉRENCES PUBLIQUES

☑ Sources

1 <https://github.com/jordanvalnet/exalthon26/issues/1>

2 <https://github.com/jordanvalnet/exalthon26/pull/2>